Led blinking:

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX

ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL

JMP XX
XX:

MOV AL,0000H
MOV DX,PORTA
OUT DX,AL
MOV CX,0DF36H
XXX:
LOOP XXX

LOOPY1:
MOV AL,00FFH
MOV DX,PORTA
OUT DX,AL
MOV CX,0DF36H

LOOPY2:LOOP LOOPY2
JMP XX
CODE ENDS
END
```

1 Push button on-off:

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX

ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL

JMP XX
XX:

MOV DX,PORTB
IN AL,DX
CMP AL,1
JE ON
MOV AL,0
MOV DX,PORTA
OUT DX,AL
JMP XX
ON:
MOV AL,1
MOV DX,PORTA
OUT DX,AL
JMP XX



CODE ENDS
END
```

2 push button on-off:

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX

ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL

JMP XX
XX:

MOV DX,PORTC
IN AL,DX
MOV BL,AL
MOV DX,PORTC
IN AL,DX
MOV CL,AL
CMP BL,10000000B
JE ON
CMP CL,01000000B
JE OFF
JMP XX
ON:
MOV AL,0FFH
MOV DX,PORTA
OUT DX,AL
JMP XX
OFF:
MOV AL,0
MOV DX,PORTA
OUT DX,AL
JMP XX
CODE ENDS
END
```

2 push button(xor operation):

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX

ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL

JMP XX
XX:

MOV DX,PORTC
IN AL,DX
MOV BL,AL
MOV DX,PORTC
IN AL,DX
MOV CL,AL
CMP BL,10000000B
JE ON
CMP CL,01000000B
JE OFF
JMP XX
ON:
MOV AL,0FFH
MOV DX,PORTA
```

```
OUT DX,AL
JMP XX
OFF:
MOV AL,0
MOV DX,PORTA
OUT DX,AL
JMP XX



CODE ENDS
END
```

2 push button (blinking rate high-low):

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX

ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL

MOV BX,0DF36H
JMP XX
XX:
MOV DX,PORTB
MOV AL,0FFH
OUT DX,AL
MOV CX,BX
RUN:
LOOP RUN
MOV DX,PORTB
MOV AL,0H
OUT DX,AL
MOV CX,BX
RUN2:
LOOP RUN2
MOV DX,PORTA
IN AL,DX
MOV CL,AL
MOV DX,PORTA
IN AL,DX
MOV CH,AL
CMP CL,00000010B
JE HH
CMP CH,00000100B
JE LL
JMP XX
HH:
ADD BX,10000
JMP XX
LL:
SUB BX,10000
JMP XX



CODE ENDS
END
```

1 push button(debounce):

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX
```

```
ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL
MOV BL,0B
JMP XX
XX:
MOV DX,PORTB
MOV AL,BL
OUT DX,AL
MOV DX,PORTA
IN AL,DX
CMP AL,00000010B
JE RECHECK
JMP XX

RECHECK:
MOV CX,2200
CHECK:
LOOP CHECK
MOV DX,PORTA
IN AL,DX
CMP AL,00000010B
JE DEBOUNCE
JMP XX
DEBOUNCE:
XOR BL,00000010B
JMP XX



CODE ENDS
END
```

2 push button (XNOR):

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX

ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL

JMP XX
XX:

MOV DX,PORTC
IN AL,DX
MOV BL,AL
MOV DX,PORTC
IN AL,DX
MOV CL,AL
CMP BL,11000000B
JE ON
CMP BL,0B
JE ON
CMP BL,10000000B
JE OFF
CMP CL,01000000B
JE OFF
JMP XX
ON:
MOV AL,0FFH
MOV DX,PORTA
OUT DX,AL
```

```
JMP XX
OFF:
MOV AL,0
MOV DX,PORTA
OUT DX,AL
JMP XX



CODE ENDS
END
```

2 push button-2 LED:

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX

ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL

JMP XX
XX:

MOV DX,PORTC
IN AL,DX
MOV BL,AL
MOV DX,PORTC
IN AL,DX
MOV CL,AL
CMP BL,01000000B
JE ON1
CMP BL,11000000B
JE ON1
JNE OFF1
A1:
CMP BL,10000000B
JE ON2
CMP BL,11000000B
JE ON2
JNE OFF2
ON1:
MOV AL,00001000B
MOV DX,PORTA
OUT DX,AL
JMP A1
ON2:
MOV AL,00100000B
MOV DX,PORTB
OUT DX,AL
JMP XX
OFF1:
MOV AL,0B
MOV DX,PORTA
OUT DX,AL
JMP A1
OFF2:
MOV AL,0B
MOV DX,PORTB
OUT DX,AL
JMP XX



CODE ENDS
END
```

1 PUSH BUTTON 2 LED:

```
DATA SEGMENT
   PORTA EQU 00H
   PORTB EQU 02H
   PORTC EQU 04H
   PORT_CON EQU 06H

DATA ENDS
CODE SEGMENT
   MOV AX,DATA
   MOV DS,AX

ORG 0000H
START:

MOV DX, PORT_CON
MOV AL,10000000B
OUT DX,AL

JMP XX
XX:

MOV DX,PORTB
IN AL,DX
CMP AL,1B
JE ON
MOV AL,0B
MOV DX,PORTA
OUT DX,AL
MOV AL,00010000B
MOV DX,PORTC
OUT DX,AL
JMP XX
ON:
MOV AL,1B
MOV DX,PORTA
OUT DX,AL
MOV AL,0B
MOV DX,PORTC
OUT DX,AL
JMP XX

CODE ENDS
END
```

Timer(OPM):

```
void TimerOPM_init()
{
        RCC->APB1ENR |= 1UL<<0;
        TIM2->CNT = 0;
// Prescaler value
        TIM2->PSC = 3600 - 1;
// Auto-reload value for 1000 ms
        TIM2->ARR = 10000 - 1;
// Configure as OPM
        TIM2->CR1 |= 1UL<<3;
}
/*while(1) {

                // Timer OPM er while loop.....

        TIM2->CR1 |= 1; // Start Timer
        while(TIM2->CR1 & 1); // Check if still counting
        GPIOA->ODR ^= 1UL<<0;  // Toggle output
        }*/
```

Timer(OCM):

```
void TimerOCM_init()
{
        RCC->APB1ENR |= (1UL << 0);   // TIM2 clock enable
        // Configure TIM2
        // Prescaler value (PSC)
        TIM2->PSC = 3600 - 1;
        // Auto-reload value (ARR) for 1 second timing (assuming 36MHz clock)
        TIM2->ARR = 10000 - 1;
```

```
        // Set the capture/compare register value for
CH1 (50% duty cycle)
        TIM2->CCR1 = 5000 - 1;
        // Set Output Compare mode to Toggle (011)
        TIM2->CCMR1 |= 3<<4;
        // Enable the output on CH1
        TIM2->CCER |= 1<<0;
        TIM2->CR1 |= 1;


}
/*while(1)
        {
                        // Timer OCM er while
loop.....
        }*/
```